Question 1: Multiple Linear Regression and Coefficient Interpretation A real-estate company wants to predict house prices using area, number of bedrooms, house age, and location score.

The trained model is:

Price = 12 + 0.045 * Area + 6 * Bedrooms - 1.2 * Age + 8 * Location_Score

```
# 1..Calculate the predicted price for each house using the given equation
1. House A
    price=90
2.House B
    Price=137
3. House C
    price=174.4
4. House D
    price=99.1
```

```
# 2.Interpret all four feature coefficients while holding the other features constant
1. Area (+0.045): If area increases by one unit, house price increases by 0.045 units, keeping ot
2. Bedrooms (+6): If the number of bedrooms increases by one, house price increases by 6 units, k
3. Age (-1.2): If house age increases by one year, house price decreases by 1.2 units, keeping ot
4. Location Score (+8): If location score increases by one point, house price increases by 8 unit
```

```
# 3..Suppose the area coefficient changes from 0.045 to 0.060. Calculate the change in predicted
old area coefficient  0.045
  house B
    price =137
new area coefficient 0.060
  house B
   price =164

change in predicted price
 change =new -old
        =164-137
        =27
```

```
# 4.Explain why a larger coefficient does not necessarily mean a feature is more important when f
Different features use different measurement scales and units.
Coefficients depend on the measurement scale of each feature.
Therefore, a larger coefficient does not always indicate greater feature importance.
We can use Standardization to bring features onto a common scale and compare their standardized c
```

Question 2: MSE Calculation Given the following actual and predicted values, calculate the Residuals, Squared Errors, SSE, and MSE:

```
Step 1: Calculate Residuals
Formula:
      Residual = Actual - Predicted
A.2
B.-3
c.2
D.-4
E.3

step2: calculate squared error
      squared_error=(Residual)^2

a -> 4
```

```
b -> 9
c ->4
d -> 16
e ->9

step 3 : claculate SSE - sum of squared error
    SSE =42

step 4: calculate MSE - mean of suqared error
  MSE = 8.4

The Mean Squared Error (MSE) measures the average squared difference between actual and predicted
The calculated MSE is 8.4, which indicates the average squared prediction error is 8.4 square uni
A lower MSE indicates better prediction performance.
MSE penalizes larger prediction errors more heavily because errors are squared.
```

Question 3: Data Science Project - House Price Prediction Create or use a dataset containing at least 20 houses with the following columns:

```
import pandas as pd
import numpy as nn
data={
  "Area": [800, 1000, 1200, 1500, 1800,
             2000, 2200, 2500, 2800, 3000,
             900, 1100, 1400, 1600, 1900,
             2100, 2300, 2600, 2900, 3200],
    "Bedrooms": [2, 2, 3, 3, 3,
                 4, 4, 4, 4, 5,
                 2, 2, 3, 3, 3,
                 4, 4, 4, 5, 5],
    "Bathrooms": [1, 1, 2, 2, 2,
                  2, 3, 3, 3, 3,
                  1, 2, 2, 2, 2,
                  3, 3, 3, 3, 4],
    "Age": [5, 8, 3, 10, 6,
            4, 7, 2, 5, 3,
            12, 9, 6, 11, 8,
            5, 4, 7, 3, 2],
    "House_Price": [2500000, 3000000, 3500000, 4200000, 4800000,
                    5500000, 6000000, 6800000, 7200000, 8000000,
                    2700000, 3200000, 3900000, 4500000, 5000000,
                    5800000, 6300000, 7000000, 7600000, 8500000]
}

df=pd.DataFrame(data)
```

```
# 1. Perform EDA
#  rows aur columns .
df.shape
```

```
(20, 5)
```

```
# Data types .
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 5 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   Area         20 non-null     int64
 1   Bedrooms     20 non-null     int64
 2   Bathrooms    20 non-null     int64
 3   Age          20 non-null     int64
 4   House_Price  20 non-null     int64
```

```
dtypes: int64(5)
memory usage: 932.0 bytes
```

```
# Missing values.
df.isnull().sum()
```

	0
Area	0
Bedrooms	0
Bathrooms	0
Age	0
House_Price	0

dtype: int64

```
# duplicate value
df.duplicated().sum()
```

```
np.int64(0)
```

```
df.columns
```

```
Index(['Area', 'Bedrooms', 'Bathrooms', 'Age', 'House_Price'], dtype='object')
```

```
# feature and target variable
x=df[['Area', 'Bedrooms', 'Bathrooms', 'Age']]
y=df['House_Price']
```

```
# dataset ki average,min,max values
print(f'\n{df.mean()}')
print(f'\n{df.max()}')
print(f'\n{df.min()}')
```

```
Area              1940.00
Bedrooms             3.45
Bathrooms            2.35
Age                  6.00
House_Price    5300000.00
dtype: float64

Area              3200
Bedrooms             5
Bathrooms            4
Age                 12
House_Price    8500000
dtype: int64

Area               800
Bedrooms             2
Bathrooms            1
Age                  2
House_Price    2500000
dtype: int64
```

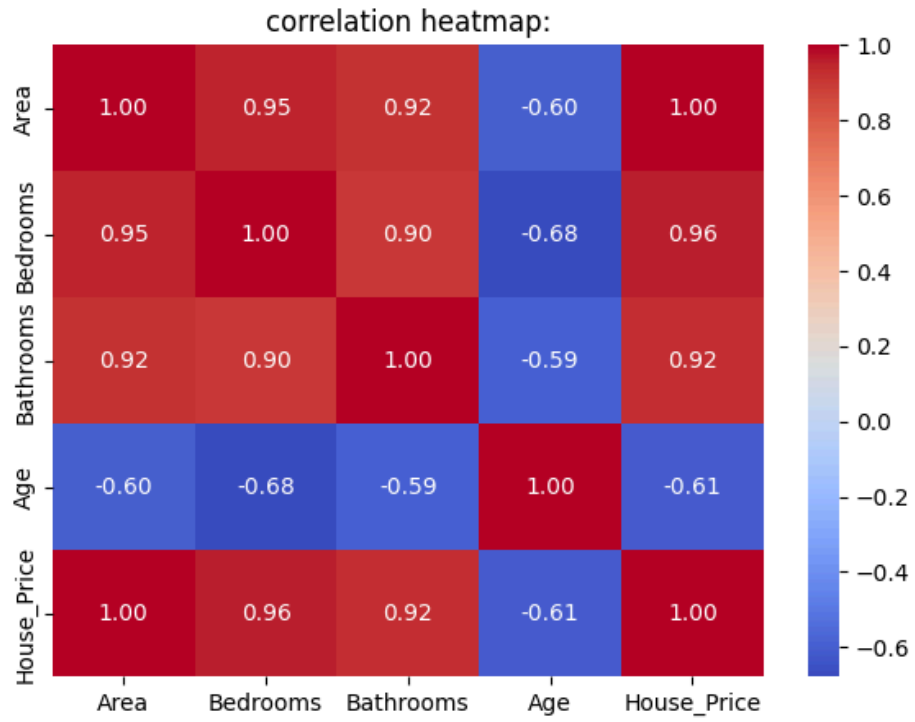
```
# relationship b/w features
correlation=df.corr()
print(correlation)
```

```
                Area  Bedrooms  Bathrooms       Age  House_Price
Area        1.000000  0.953341   0.916954 -0.601552     0.998553
Bedrooms    0.953341  1.000000   0.898097 -0.677524     0.959920
```

```
Bathrooms     0.916954  0.898097    1.000000 -0.591545     0.924042
Age          -0.601552 -0.677524   -0.591545  1.000000    -0.611124
House_Price   0.998553  0.959920    0.924042 -0.611124     1.000000
```

```
 # heatmap use krenge
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(7,5))
sns.heatmap(correlation,annot=True,cmap='coolwarm',fmt='.2f')      # red=positive
plt.title('correlation heatmap:')
plt.show()                                                              # light red=positive mode
                                                                  # blue=negative
```




```
# 3. Split the data into train and test sets
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(
    x,y,random_state=42,test_size=0.2
)
```

```
# 4. Train a Multiple Linear Regression model.
from sklearn.linear_model import LinearRegression
model=LinearRegression()
model.fit(x_train,y_train)
print('model trained successfully')
```

```
model trained successfully
```

```
# 5. Calculate MAE, MSE, RMSE, and R².
# 1.MAE calculate
from sklearn.metrics import mean_absolute_error,r2_score
y_pred=model.predict(x)

MAE=mean_absolute_error(y,y_pred)

# 2.mse calculate
from sklearn.metrics import mean_squared_error
MSE=mean_squared_error(y,y_pred)
```

```
# 3..r2_score calculate
from sklearn.metrics import r2_score
r2=r2_score(y,y_pred)

#4.. calculta rmse
import numpy as np
rmse=np.sqrt(MSE)


print(MAE)
print(MSE)
print(rmse)
print(r2)
```

```
74997.99701490614
7684810585.616625
87663.05142770598
0.997636897113894
```

```
# 6. Interpret the coefficients
coefficient=model.coef_
intercept=model.intercept_
print(coefficient ,'---->',
      intercept)
```

```
[  2124.52235267 197713.23714697 146559.97001984   5537.93072256] ----> 97895.70262778457
```

```
# 7. Predict the price of a new house.
new_house=({
    "Area": [2000],
    "Bedrooms": [3],
    "Bathrooms": [2],
    "Age":[5]
})

df=pd.DataFrame(new_house)
predicted_price=model.predict(df)
print(f'MY new house predicted_price {predicted_price}')
```

```
MY new house predicted_price [5260889.71305623]
```

```
# 8. Explain which features appear useful for prediction

Area: It is an important feature because larger houses generally have higher prices.
Bedrooms: The number of bedrooms can affect the house price.
Bathrooms: More bathrooms may increase the value of a house.
Age: Older houses may have lower prices due to depreciation.
```

Question 4: Data Science Project - Salary Prediction Create or use a dataset containing at least 20 employees with: • Years of Experience • Education Score • Skill Score • Salary

```
# 1. Perform basic EDA
data={"Years_of_Experience": [
        1, 2, 3, 4, 5,
        6, 7, 8, 9, 10,
        2, 3, 4, 5, 6,
        7, 8, 9, 10, 12
    ],

    "Education_Score": [
        50, 55, 60, 65, 70,
        75, 80, 85, 90, 95,
        60, 65, 70, 75, 80,
        85, 90, 95, 80, 95
```

```
    ],

    "Skill_Score": [
        45, 50, 55, 60, 65,
        70, 75, 80, 85, 90,
        55, 60, 65, 70, 75,
        80, 85, 90, 95, 98
    ],

    "Salary": [
        20000, 25000, 30000, 35000, 40000,
        45000, 50000, 55000, 60000, 65000,
        28000, 33000, 38000, 43000, 48000,
        53000, 58000, 63000, 68000, 75000
    ]
}
df=pd.DataFrame(data)
```

```
df.shape
```

```
(20, 4)
```

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
df.isnull().sum()
```

	0
Years_of_Experience	0
Education_Score	0
Skill_Score	0
Salary	0

dtype: int64

```
df.dtypes
```

	0
Years_of_Experience	int64
Education_Score	int64
Skill_Score	int64
Salary	int64

dtype: object

```
correlation=df.corr()
print(correlation)
```

```
                    Years_of_Experience  Education_Score  Skill_Score  \
Years_of_Experience             1.000000         0.942672     0.984899
Education_Score                 0.942672         1.000000     0.953559
Skill_Score                     0.984899         0.953559     1.000000
Salary                          0.995099         0.951258     0.997010

                      Salary
Years_of_Experience  0.995099
Education_Score      0.951258
```

```
Skill_Score          0.997010
Salary               1.000000
```

```
print(f'\n{df.mean()}')
print(f'\n{df.max()}')
print(f'\n{df.min()}')
```

```
Years_of_Experience        6.05
Education_Score           76.00
Skill_Score               72.40
Salary                 46600.00
dtype: float64

Years_of_Experience       12
Education_Score           95
Skill_Score               98
Salary                 75000
dtype: int64

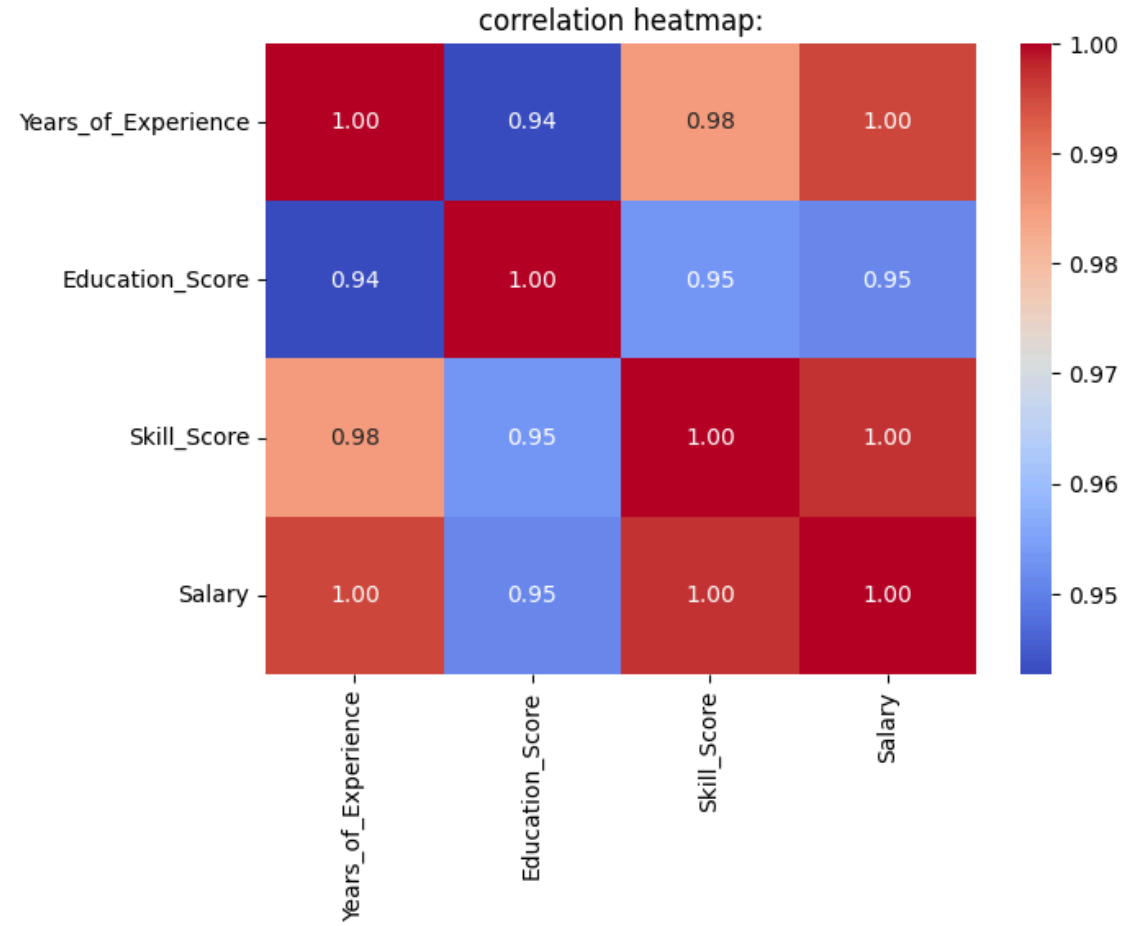
Years_of_Experience        1
Education_Score           50
Skill_Score               45
Salary                 20000
dtype: int64
```

```
df.columns
```

```
Index(['Years_of_Experience', 'Education_Score', 'Skill_Score', 'Salary'], dtype='object')
```

```
x=df[['Years_of_Experience', 'Education_Score', 'Skill_Score']]
y=df['Salary']
```

```
plt.figure(figsize=(7,5))
sns.heatmap(correlation,annot=True,cmap='coolwarm',fmt='.2f')      # red=positive
plt.title('correlation heatmap:')
plt.show()
```




```
# 2. Check for missing values and outliers
for col in df.columns:
  Q1=df[col].quantile(0.25)
  Q3=df[col].quantile(0.75)

  IQR=Q3-Q1

  lower=Q1-1.5*IQR
  upper=Q3+1.5*IQR

  outliers=df[(df[col]<lower) | (df[col]>upper)]

  print(f'outliers is {len(outliers),col}')
```

```
outliers is (0, 'Years_of_Experience')
outliers is (0, 'Education_Score')
outliers is (0, 'Skill_Score')
outliers is (0, 'Salary')
```

```
x_train,x_test,y_train,y_test=train_test_split(
    x,y,random_state=42,test_size=0.2
)
```

```
y_pred=model.predict(x)
```

```
#3 . Train a Multiple Linear Regression model.
model=LinearRegression()
model.fit(x,y)
print('model trained successfully')
```

```
model trained successfully
```

```
# 4. Calculate MSE and RMSE
# calculate MSE
Mse=mean_squared_error(y,y_pred)
RMSE=np.sqrt(Mse)

print(Mse,'------>',RMSE)
```

```
44625.43311163309 ------> 211.2473268745266
```

```
# 5. Interpret the model coefficients
print(model.coef_)
print(model.intercept_)
```

```
[2206.57931419  -12.27647457  576.57612808]
-7560.904456569406
```

```
# 6. Predict salary for a new employee
new_emp=({
    "Years_of_Experience": [5],
    "Education_Score": [80],
    "Skill_Score": [85]
})

df2=pd.DataFrame(new_emp)
predicted_salary=model.predict(df2)
print(f'predict_salary of new_emp {predicted_salary}')
```

```
predict_salary of new_emp [51498.84503564]
```

```
# 7. Compare training and testing performance
y_train_pred = model.predict(x_train)     # Training predictions

y_test_pred = model.predict(x_test)      # Testing predictions
train_r2 = r2_score(y_train, y_train_pred)    # Training performance
train_mae = mean_absolute_error(y_train, y_train_pred)

test_r2 = r2_score(y_test, y_test_pred)    # Testing performance
test_mae = mean_absolute_error(y_test, y_test_pred)

print("Training R2:", train_r2)
print("Testing R2:", test_r2)

print("Training MAE:", train_mae)
print("Testing MAE:", test_mae)
```

```
Training R2: 0.9997061913188069
Testing R2: 0.9999983822967913
Training MAE: 167.84569277948003
Testing MAE: 21.058186307700453
```

Question 5: Data Science Mini Project - Sales Prediction Create a dataset containing at least 25 observations with: • Advertising Spend • Discount • Number of Customers • Sales

```
data = {
    "Advertising_Spend": [
        1000, 1500, 2000, 2500, 3000,
        3500, 4000, 4500, 5000, 5500,
        1200, 1800, 2200, 2800, 3200,
        3800, 4200, 4800, 5200, 5800,
        1300, 2300, 3300, 4300, 5300
    ],
    "Discount": [
```

```
        5, 10, 5, 15, 10,
        20, 15, 10, 20, 25,
        5, 10, 15, 10, 20,
        15, 20, 25, 15, 20,
        10, 15, 20, 10, 25
    ],
    "Number_of_Customers": [
        50, 60, 70, 80, 90,
        100, 110, 120, 130, 140,
        55, 65, 75, 85, 95,
        105, 115, 125, 135, 145,
        58, 78, 98, 118, 138
    ],
    "Sales": [
        10000, 14000, 18000, 22000, 26000,
        30000, 34000, 38000, 42000, 46000,
        11500, 15500, 19500, 23500, 27500,
        31500, 35500, 39500, 43500, 47500,
        12000, 20000, 28000, 36000, 44000
    ]
}

df3=pd.DataFrame(data)
```

```
df3.shape
```

```
(25, 4)
```

```
df3.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 25 entries, 0 to 24
Data columns (total 4 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   Advertising_Spend    25 non-null     int64
 1   Discount             25 non-null     int64
 2   Number_of_Customers  25 non-null     int64
 3   Sales                25 non-null     int64
dtypes: int64(4)
memory usage: 932.0 bytes
```

```
df3.isnull().sum()
```

	0
Advertising_Spend	0
Discount	0
Number_of_Customers	0
Sales	0

dtype: int64

```
df3.dtypes
```

	0
Advertising_Spend	int64
Discount	int64
Number_of_Customers	int64
Sales	int64

dtype: object

```
correlation=df3.corr()
print(correlation)
```

```
                     Advertising_Spend  Discount  Number_of_Customers  \
Advertising_Spend             1.000000  0.730003             0.999376
Discount                      0.730003  1.000000             0.733691
Number_of_Customers           0.999376  0.733691             1.000000
Sales                         0.999563  0.728299             0.999263

                        Sales
Advertising_Spend    0.999563
Discount             0.728299
Number_of_Customers  0.999263
Sales                1.000000
```

```
print(f'\n{df3.mean()}')
print(f'\n{df3.max()}')
print(f'\n{df3.min()}')
```

```

Advertising_Spend       3360.0
Discount                  14.8
Number_of_Customers       97.6
Sales                  28600.0
dtype: float64

Advertising_Spend       5800
Discount                  25
Number_of_Customers      145
Sales                  47500
dtype: int64

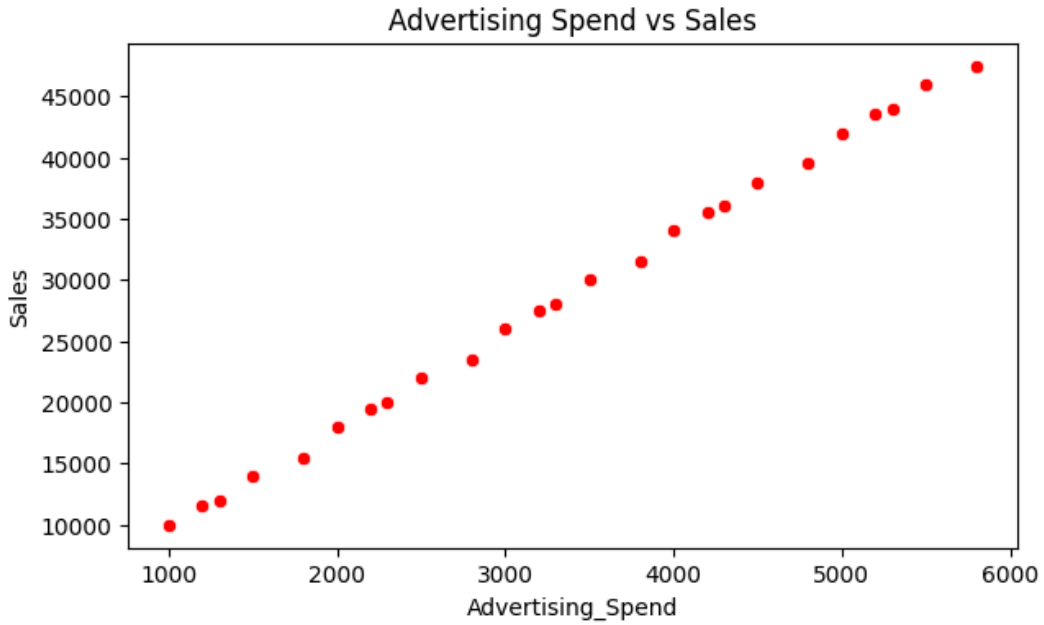
Advertising_Spend       1000
Discount                   5
Number_of_Customers       50
Sales                  10000
dtype: int64
```

```
df3.columns
```

```
Index(['Advertising_Spend', 'Discount', 'Number_of_Customers', 'Sales'], dtype='object')
```

```
x=df3[['Advertising_Spend', 'Discount', 'Number_of_Customers']]
y=df3['Sales']
```

```
# 2. Analyze relationships between features and Sales.
plt.figure(figsize=(7,4))
sns.scatterplot(data=df3,x='Advertising_Spend',y='Sales',color='red')
plt.title("Advertising Spend vs Sales")
plt.xlabel('Advertising_Spend')
plt.show()
```




```
# 3. Train a Multiple Linear Regression model using multiple features
x=df3[['Advertising_Spend', 'Discount', 'Number_of_Customers']]
y=df3['Sales']
x_train,x_test,y_train,y_test=train_test_split(x,y,random_state=42,test_size=0.2)
model=LinearRegression()
model.fit(x,y)

print('model train successfuly')
```

```
model train successfuly
```

```
#4.  Calculate MSE, RMSE, MAE, and R^2
# 1.MAE calculate
y_pred=model.predict(x)

MAE=mean_absolute_error(y,y_pred)

# 2.mse calculate
MSE=mean_squared_error(y,y_pred)

# 3..r2_score calculate
r2=r2_score(y,y_pred)

#4.. calculta rmse
rmse=np.sqrt(MSE)


print(MAE)
print(MSE)
print(rmse)
print(r2)
```

```
295.805426567925
102266.62674133255
319.79153638164433
0.9992237237988361
```

```
# 5. Analyze residuals.
residual=y-y_pred

print(residual)
```

```
0     235.738835
1     284.518505
```

```
2     230.746678
3     330.802097
4     277.030270
5     377.085689
6     323.313862
7     269.542036
8     369.597454
9     418.377125
10     23.595872
11   -505.834611
12    121.155213
13   -510.826767
14    167.438805
15   -464.543175
16    162.446648
17   -366.983835
18    106.178743
19   -423.251740
20   -336.772126
21   -290.488534
22   -244.204942
23   -351.748596
24   -202.913507
Name: Sales, dtype: float64
```

```
# 6. Identify observations with unusually large errors, if any.
y_test_pred = model.predict(x_test)
error_df = pd.DataFrame({
    "Actual_Sales": y_test,
    "Predicted_Sales": y_test_pred
})

# Calculate absolute error
error_df["Absolute_Error"] = abs(
    error_df["Actual_Sales"] - error_df["Predicted_Sales"]
)

error_df = error_df.sort_values(
    by="Absolute_Error", ascending=False
)

print(error_df)
```

```
    Actual_Sales  Predicted_Sales  Absolute_Error
```